Large Language Models: From Foundations to Production

An industry-level course for students · 16 modules · about 14 weeks

How to Use This Course

Who it is for: students and early-career engineers who know Python and want to understand LLMs deeply and build production systems with them.

Prerequisites: Python (functions, classes, NumPy), basic machine learning (train/test split, loss, gradient descent), high-school-level linear algebra and probability. PyTorch experience helps, but it is taught as we go.

Tooling: Python 3.11+, PyTorch, Hugging Face (transformers, datasets, peft, trl), Jupyter or Google Colab (a free GPU is enough for most labs), Git, Docker, and an API key from any LLM provider.

Structure:

- Part 1, Foundations (Modules 1-6): how LLMs are built and trained.
- Part 2, Working with LLMs (Modules 7-12): APIs, prompting, RAG, agents, fine-tuning, evaluation.
- Part 3, Production and Frontier (Modules 13-16): serving, LLMOps, safety, advanced topics.
- Part 4, Capstone: ship an end-to-end project.

Every module has learning goals, core concepts, a hands-on lab and a self-check. Do the labs. Reading alone will not give you industry skills.

Learning outcomes. By the end you can: (1) explain and implement a Transformer, (2) describe the full pipeline from pretraining to alignment, (3) build and evaluate RAG and agent systems, (4) fine-tune and quantize open models, (5) deploy, monitor and secure an LLM application.

Part 1: Foundations

Module 1: What Is a Large Language Model?

Goals: define an LLM precisely, know its history, and understand what it can and cannot do.

Core idea. An LLM is a neural network trained to predict the next token given all previous tokens. It models P(x_t | x_1, ..., x_t-1). Generating text means sampling one token, appending it, and repeating. This is called autoregressive generation. Everything else (chat, code, reasoning) is built on top of this single mechanism.

Why 'large'. Billions of parameters, trillions of training tokens, and enormous compute. At scale, models show in-context learning (learning a task from examples in the prompt), instruction following and multi-step reasoning without being explicitly programmed for them.

Short history:

- Statistical language models: n-grams, smoothing.
- Word embeddings (word2vec, GloVe, ~2013): words become vectors.
- RNNs and LSTMs, then sequence-to-sequence with attention (2014-2016).
- The Transformer, 'Attention Is All You Need' (2017): parallel training, no recurrence.
- Pretrained models: BERT (encoder), GPT-2 and GPT-3 (decoder). Scaling becomes the main recipe.
- Instruction tuning and RLHF (2022): base models become assistants.
- Open-weight models (Llama, Mistral, Qwen, DeepSeek and others), multimodal models, reasoning models, agents.

Landscape. Closed models are used through APIs. Open-weight models can be downloaded, fine-tuned and self-hosted. Choosing between them is a core industry decision: quality, cost, latency, privacy, control.

Limits you must internalize: hallucination (fluent but false output), knowledge cutoff, finite context window, sensitivity to prompt wording, non-determinism, and no built-in access to your private data.

Lab 1. Inspect next-token probabilities of a small model:

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

tok = AutoTokenizer.from_pretrained('gpt2')
model = AutoModelForCausalLM.from_pretrained('gpt2')
ids = tok('The capital of France is', return_tensors='pt')
with torch.no_grad():
    logits = model(**ids).logits[0, -1]
probs = torch.softmax(logits, dim=-1)
top = torch.topk(probs, 5)
for p, i in zip(top.values, top.indices):
    print(repr(tok.decode(i)), round(p.item(), 3))
```

Self-check: Why is an LLM 'just' next-token prediction yet able to answer questions? What is the difference between a base model and a chat model?

Module 2: Math and ML Prerequisites

Goals: gain the minimum math needed to read any LLM paper.

Linear algebra. Vectors, matrices, dot product, matrix multiplication and shape rules, norms, cosine similarity. Almost every LLM operation is a matrix multiplication, so always track tensor shapes.

Probability and information theory. Conditional probability and the chain rule (a sentence's probability is the product of next-token probabilities), entropy, cross-entropy, KL divergence, and perplexity (exp of average cross-entropy; lower is better).

Calculus and optimization. Gradients, the chain rule, backpropagation. Gradient descent, SGD with momentum, Adam and AdamW, learning rate, batch size, overfitting, regularization (dropout, weight decay).

Neural network building blocks. MLPs, activation functions (ReLU, GELU, SwiGLU), initialization, normalization (LayerNorm, RMSNorm), residual connections.

Two formulas to know cold:

- `softmax(z_i) = exp(z_i) / sum_j exp(z_j)` turns raw scores (logits) into probabilities.
- `loss = -log P(correct next token)` is cross-entropy, averaged over all positions.

Lab 2. Implement softmax, cross-entropy and a tiny two-layer network with manual backprop in NumPy, then verify against PyTorch autograd.

Self-check: What does a perplexity of 10 mean intuitively? Why do we subtract the max logit before softmax?

Module 3: From Text to Numbers (Tokenization, Embeddings, Positions)

Goals: understand exactly what the model sees.

Tokenization. Models do not read characters or words but tokens, which are subword pieces. Word-level vocabularies are huge and fail on unseen words; character-level sequences are too long. Subword tokenization is the compromise.

Byte Pair Encoding (BPE):

1. Start with a vocabulary of single bytes or characters.
2. Count the most frequent adjacent pair in the corpus.
3. Merge it into a new token and add it to the vocabulary.

4. Repeat until the target vocabulary size (commonly 32k-200k) is reached.

Other schemes: WordPiece (BERT), Unigram and SentencePiece (language-independent, used by many multilingual models), and byte-level BPE (never produces unknown tokens).

Practical consequences:

- You pay and are rate-limited per token, so learn to count tokens.
- Languages underrepresented in training data, such as Hindi, often need more tokens per sentence, which raises cost and uses up the context window faster.
- Odd behavior with numbers, spelling and whitespace often traces back to tokenization.
- Special tokens (beginning of sequence, end of sequence, role markers) and chat templates must match what the model was trained on. A wrong template silently degrades quality.

Embeddings. Each token ID indexes a row of a learned matrix of shape (vocab_size, d_model), giving a dense vector. Similar usage leads to nearby vectors. Many models tie the input embedding matrix with the output projection to save parameters.

Positional information. Attention alone has no notion of order, so position must be injected:

- Sinusoidal and learned absolute positions (original Transformer, GPT-2).
- Relative approaches: **RoPE** (rotary embeddings, now the standard in most open models) and ALiBi.
- Context extension techniques (position interpolation, YaRN) let a model run on sequences longer than it was trained on.

Lab 3. Implement a 100-line BPE trainer. Then compare how several tokenizers split the same English and Hindi paragraph and compute the cost difference.

Self-check: Why can't a model easily count letters in a word? What happens if you use the wrong chat template at inference?

Module 4: The Transformer Architecture

Goals: be able to draw, explain and implement a decoder-only Transformer.

Self-attention. Each token produces a query (Q), key (K) and value (V) vector. A token's output is a weighted average of all values, where the weights come from how well its query matches each key:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

Dividing by √d_k keeps the dot products from growing with dimension and saturating the softmax. In decoder models a **causal mask** blocks attention to future tokens, so training can predict every position in parallel.

Multi-head attention. Run several attention operations in parallel on smaller subspaces so different heads can specialize (syntax, coreference, copying), then concatenate and project.

The Transformer block:

1. Normalization (pre-norm), then multi-head attention, then a residual add.
2. Normalization, then a position-wise feed-forward network (typically 4x expansion, often SwiGLU), then a residual add.

Stack N such blocks, add a final norm and a linear 'LM head' that outputs logits over the vocabulary. A rough parameter count is 12 x layers x d_{model}^2 (plus embeddings).

Three families:

- Encoder-only (BERT): bidirectional, best for classification and embeddings.
- Decoder-only (GPT, Llama, Claude-style models): causal, best for generation. The dominant design today.
- Encoder-decoder (T5, original Transformer): translation and summarization.

Minimal causal self-attention in PyTorch:

```
import torch, torch.nn as nn, torch.nn.functional as F

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=False)
        self.out = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x):                       # x: (B, T, C)
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=-1)
        q, k, v = (t.view(B, T, self.h, self.dk).transpose(1, 2) for t in (q,
k, v))
        y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        return self.out(y.transpose(1, 2).reshape(B, T, C))
```

Modern improvements used in industry:

- **RMSNorm and SwiGLU** replace LayerNorm and plain ReLU/GELU MLPs.

- **KV cache:** during generation, store past keys and values so each new token costs one step instead of recomputing everything. It dominates inference memory.
- **MQA and GQA** (multi-query and grouped-query attention): share K/V heads across query heads to shrink the KV cache.
- **FlashAttention:** an exact algorithm that tiles the computation to avoid writing the full n x n attention matrix to GPU memory, giving large speed and memory wins.
- **Sliding-window and sparse attention** reduce the $O(n^2)$ cost for long contexts.
- **Mixture of Experts (MoE):** replace the FFN with many expert FFNs and a router that sends each token to the top-k. Total parameters are large, but active parameters per token are small.

Lab 4. Build a nanoGPT-style model (embeddings, N blocks, LM head) and train it on a small text corpus such as Tiny Shakespeare. Generate samples and plot the loss curve.

Self-check: Why is attention $O(n^2)$ in sequence length? Why does GQA help inference more than training? What is the KV cache and why does it matter?

Module 5: Pretraining at Scale

Goals: understand how a base model is produced and what it costs.

Objective. Causal language modeling: minimize cross-entropy of the next token over a massive corpus. (BERT-style models use masked language modeling instead.)

Data is the product. Typical pipeline:

1. Collect: web crawls, books, code, scientific papers, multilingual text.
2. Extract and clean text, identify language.
3. Quality filtering (heuristics and classifier-based).
4. Deduplication (exact and fuzzy, for example MinHash), which reduces memorization and improves quality.
5. Remove PII and toxic content where required, and decontaminate against benchmark test sets.
6. Choose a data mixture (web, code, math, multilingual) and train for a chosen number of tokens.

Data quality often matters more than raw quantity, and synthetic data is increasingly part of the mix.

Scaling laws. Loss falls predictably as a power law in parameters, data and compute. The Chinchilla result suggests roughly 20 training tokens per parameter is compute-optimal; in practice, teams often train smaller models on far more tokens because a smaller model is cheaper to serve. A handy estimate: training compute ≈ 6 x parameters x tokens FLOPs.

Training recipe:

- AdamW optimizer, linear warmup then cosine (or similar) learning-rate decay, gradient clipping.
- bf16 mixed precision, gradient accumulation, activation checkpointing to save memory.
- Large global batch sizes, careful monitoring of loss spikes and divergence, frequent checkpoints.

Distributed training. One GPU cannot hold or train a large model, so work is split:

- **Data parallelism (DP):** replicate the model, split the batch.
- **ZeRO and FSDP:** shard optimizer state, gradients and parameters across GPUs.
- **Tensor parallelism:** split individual matrix multiplications across GPUs.
- **Pipeline parallelism:** split layers into stages.
- **Sequence/context and expert parallelism** for long contexts and MoE.

Real systems combine these (3D parallelism) and depend on fast interconnects and fault-tolerant tooling.

Lab 5. Train a small model with mixed precision and gradient accumulation; compute your training FLOPs and tokens-per-second; then run a two-GPU or simulated FSDP example from the PyTorch docs.

Self-check: Why does deduplication help? What does Chinchilla-optimal mean, and why do companies deviate from it?

Module 6: Post-Training and Alignment

Goals: understand how a base model becomes a helpful, safer assistant.

Why it is needed. A base model continues text; it does not follow instructions. Post-training teaches it to.

Step 1: Supervised fine-tuning (SFT) / instruction tuning. Train on (instruction, ideal response) pairs formatted with the chat template. Compute loss only on response tokens, not on the prompt. A small set of high-quality examples goes a long way.

Step 2: Preference optimization.

- **RLHF:** humans rank candidate responses, a reward model is trained on those rankings, and the policy is optimized (classically with PPO) to maximize reward while a KL penalty keeps it close to the SFT model.
- **DPO** and relatives (IPO, KTO, ORPO): optimize directly on preference pairs with a simple loss, with no separate reward model and no RL loop. Popular because it is simpler and stable.

- **RLAIF and Constitutional AI:** use an AI model guided by written principles to produce preference labels, reducing human labeling cost.

Step 3: Reinforcement learning with verifiable rewards. For math, code and logic, correctness can be checked automatically. Algorithms such as GRPO train models to produce long reasoning traces and reward correct final answers. This is the basis of modern 'reasoning' models, which spend more compute at inference time (test-time compute) to think before answering.

Other techniques: rejection sampling (generate many answers, keep the best), distillation (train a small model on a large model's outputs), and synthetic data generation.

Failure modes: reward hacking (exploiting the reward signal), sycophancy (agreeing with the user), over-refusal, and loss of capability ('alignment tax'). Good teams measure these explicitly.

Lab 6. Fine-tune a small open model with SFT using the TRL library, then run DPO on a public preference dataset. Compare outputs before and after.

Self-check: Why mask the prompt tokens in SFT? What does the KL penalty prevent in RLHF? How does DPO avoid training a reward model?

Part 2: Working with LLMs

Module 7: Using LLM APIs and Controlling Generation

Goals: call models reliably, control their output, and reason about cost and latency.

Anatomy of a request. You send a model name, a list of messages with roles (system, user, assistant), and generation settings such as the maximum output tokens. The model returns text plus usage data. The API is stateless: you resend the whole conversation every turn.

```
import anthropic

client = anthropic.Anthropic()            # reads ANTHROPIC_API_KEY from the
environment
resp = client.messages.create(
    model='claude-sonnet-5-5',            # use any current model name
    max_tokens=500,
    system='You are a concise technical tutor.',
    messages=[{'role': 'user', 'content': 'Explain the KV cache in 3
sentences.'}],
)
print(resp.content[0].text)
print(resp.usage)                         # input and output token counts
```

The same pattern applies to every provider, and to open models served behind an OpenAI-compatible endpoint.

Decoding and sampling. The model outputs a probability distribution; you choose how to pick from it:

- **Greedy:** always take the most likely token. Deterministic but can be repetitive.
- **Temperature:** divides logits before softmax. Low (0-0.3) for extraction and code, higher (0.7-1.0) for creative work.
- **Top-k and top-p (nucleus):** sample only from the k most likely tokens, or the smallest set whose probability sums to p.
- **Stop sequences, repetition penalties, max tokens** bound the output. Even at temperature 0, outputs can vary slightly across runs, so never assume perfect determinism.

Cost and limits. Cost = input tokens x input price + output tokens x output price. Output tokens are usually more expensive and slower. Know your context window, rate limits, and tools that cut cost: prompt caching for repeated prefixes, batch APIs for offline jobs, and routing easy queries to smaller models.

Engineering basics: streaming responses for better perceived latency, async calls, timeouts, retries with exponential backoff and jitter, and never putting API keys in source code.

Structured outputs. Applications need machine-readable results. Options: ask for JSON and validate with Pydantic, use provider schema or tool-based structured output, or use constrained decoding (grammar-guided generation) with open models. Always validate and have a retry or repair path.

Lab 7. Build a command-line chatbot with conversation history, streaming, a token and cost counter, and a JSON-extraction command validated with Pydantic.

Self-check: Why does a long conversation get more expensive each turn? When would you set temperature to 0?

Module 8: Prompt Engineering

Goals: write prompts that are clear, testable and maintainable.

Principles:

- Be explicit about the task, audience, constraints and output format.
- Give context the model cannot guess, and say what to do when information is missing.
- Show, do not just tell: examples are the strongest steering tool.
- Separate instructions from data using delimiters or XML-style tags so the model does not confuse them.

Core techniques:

- **Zero-shot and few-shot:** instructions alone versus instructions plus 2-5 representative examples (include edge cases).
- **System prompts and roles:** set persistent behavior, tone and rules.
- **Chain-of-thought:** let the model reason step by step before the final answer, which helps on math, logic and multi-step tasks. With dedicated reasoning models, state the goal and constraints and let the model do its own thinking instead of dictating each step.
- **Self-consistency:** sample several reasoning paths and take the majority answer.
- **Decomposition and prompt chaining:** split a hard task into steps, each with its own prompt and validation.
- **ReAct and reflection:** interleave reasoning with tool calls, or have the model critique and revise its own draft.

A reliable prompt skeleton: role and goal; context and documents; step-by-step instructions; rules and constraints; examples; output format; the actual input. For long documents, place the documents first and the question last, and ask the model to quote relevant passages before answering.

Prompts are code. Store them in version control, parameterize with templates, attach a test set, and review changes like pull requests. Never tune a prompt by feel on three examples.

Common pitfalls: vague instructions, contradictory rules, examples that bias the output, hidden assumptions about the model's knowledge, and ignoring prompt injection (covered in Module 15).

Lab 8. Build a support-ticket classifier and entity extractor. Create 50 labeled examples, then iterate through zero-shot, few-shot and chain-of-thought prompts, recording accuracy for each version.

Self-check: When does few-shot help more than a better instruction? Why separate instructions from untrusted data?

Module 9: Embeddings, Vector Search and RAG

Goals: build retrieval-augmented generation systems that are accurate and measurable.

Why RAG. Models have a knowledge cutoff and no access to your private documents. RAG retrieves relevant text at query time and puts it in the prompt, which grounds answers, enables citations, reduces hallucination, keeps knowledge fresh without retraining, and supports per-user access control.

Embeddings. An embedding model maps text to a vector so that semantically similar texts are close (measured by cosine similarity or dot product). Choose a model by retrieval quality on your own data, language support, dimension and cost.

The pipeline:

1. **Ingest:** parse PDFs, HTML, tables and code; clean and keep metadata (source, page, date, permissions).
2. **Chunk:** split into passages (often 200-800 tokens with some overlap), preferably along structure such as headings and paragraphs.
3. **Embed and index:** store vectors in a vector database or library (FAISS, pgvector, Qdrant, Weaviate, Milvus, Chroma). Approximate nearest-neighbor indexes such as HNSW and IVF make search fast at scale.
4. **Retrieve:** embed the query, fetch the top-k chunks.
5. **Generate:** place the chunks in the prompt, instruct the model to answer only from them, cite sources, and say 'I do not know' when the answer is absent.

```
import numpy as np
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('BAAI/bge-small-en-v1.5')
docs = ['LoRA adds low-rank adapters to frozen weights.',
        'The KV cache stores past keys and values.']
doc_vecs = embedder.encode(docs, normalize_embeddings=True)

def retrieve(query, k=2):
    q = embedder.encode([query], normalize_embeddings=True)[0]
    scores = doc_vecs @ q
    return [docs[i] for i in np.argsort(-scores)[:k]]

context = '\n'.join(retrieve('What does LoRA do?'))
prompt = ('Answer using only the context. If it is missing, say you do not
know.\n\n'
          f'Context:\n{context}\n\nQuestion: What does LoRA do?')
```

Advanced RAG techniques:

- **Hybrid search:** combine keyword search (BM25) with dense vectors, merged with reciprocal rank fusion. Exact terms such as product codes and names are where pure embeddings fail.
- **Reranking:** retrieve 50, then rescore with a cross-encoder reranker and keep the best 5.
- **Query transformation:** rewrite the question, generate multiple queries, or HyDE (embed a hypothetical answer).
- **Metadata filters, parent-document retrieval, contextual chunk enrichment.**
- **GraphRAG** for relationship-heavy corpora, and **agentic RAG** where the model decides what to search for and when.

Evaluate retrieval and generation separately. Retrieval: recall@k, MRR, nDCG on a labeled question set. Generation: faithfulness (is every claim supported by the context?), answer relevance, and citation correctness.

Typical failures: poor chunking, relevant chunk never retrieved, relevant chunk buried in the middle of a long prompt, stale indexes, and leaking documents a user should not see.

Lab 9. Build a PDF question-answering app with hybrid search, a reranker and citations. Write 30 test questions with known source pages and report recall@5 before and after adding the reranker.

Self-check: Why is hybrid search often better than dense search alone? How do you test whether a wrong answer came from retrieval or from generation?

Module 10: Tool Use and Agents

Goals: build LLM systems that act, and keep them reliable and safe.

Tool use (function calling). You describe tools with a name, description and JSON schema. The model decides when to call one and emits structured arguments. Your code executes the tool and returns the result, and the model continues. The model never runs anything itself.

```
tools = [{
    'name': 'get_weather',
    'description': 'Get current weather for a city',
    'input_schema': {'type': 'object',
                     'properties': {'city': {'type': 'string'}},
                     'required': ['city']},
}]

messages = [{'role': 'user', 'content': 'Is it raining in Delhi?'}]
while True:
    resp = client.messages.create(model=MODEL, max_tokens=1000,
                                  tools=tools, messages=messages)
    if resp.stop_reason != 'tool_use':
        break
    messages.append({'role': 'assistant', 'content': resp.content})
    results = []
    for block in resp.content:
        if block.type == 'tool_use':
            output = run_tool(block.name, block.input)    # your own function
            results.append({'type': 'tool_result',
                            'tool_use_id': block.id,
                            'content': output})
    messages.append({'role': 'user', 'content': results})
print(resp.content[0].text)
```

Workflows versus agents. A workflow follows paths you coded. An agent decides its own steps in a loop. Start with the simplest thing that works (a single call, then a chain, then a router) and add autonomy only when the task truly needs it.

Common patterns: prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, ReAct, plan-and-execute, and reflection.

Memory. Short-term memory is the context window (trim or summarize it). Long-term memory is an external store, often a vector database or a structured profile that is retrieved when needed.

Standards and frameworks. The Model Context Protocol (MCP) is an open standard for exposing tools and data to models through servers, so one integration works across many clients. Frameworks such as LangGraph, LlamaIndex and various agent SDKs help with orchestration. Learn the raw loop above first so you can debug any framework.

Making agents reliable and safe:

- Write precise tool descriptions, and validate every argument before executing.
- Set step limits, timeouts and cost caps.
- Make tools idempotent where possible and sandbox code execution.
- Require human approval for irreversible actions (payments, deletions, emails).
- Log every step so failures can be replayed.
- Treat any text a tool returns as untrusted input.

Evaluating agents: task success rate, number of steps, cost, and inspection of the full trajectory, not only the final answer.

Lab 10. Build a research agent with web search, a calculator and a file-writing tool, with a step limit, structured logs and a final cited report. Test it on 20 tasks.

Self-check: When is a fixed workflow better than an agent? What can go wrong if a tool reads an email that contains instructions?

Module 11: Fine-Tuning and Adaptation

Goals: decide when to fine-tune, and do it efficiently.

The decision ladder. Try in this order: better prompting, then RAG, then fine-tuning, and only rarely continued pretraining or training from scratch. Fine-tuning is best for consistent style and format, domain-specific behavior, tool-calling reliability, and shrinking cost and latency by distilling a large model's behavior into a small one. It is a poor way to add new facts reliably; use RAG for that.

Why parameter-efficient methods. Full fine-tuning with Adam in mixed precision needs very roughly 16 bytes per parameter for weights, gradients and optimizer states, which is out of reach for most students on large models.

PEFT methods:

- **LoRA:** freeze the base weights W and learn a low-rank update, so the effective weight is W + BA. Rank r (commonly 8-64) controls capacity, alpha scales the update. Only a tiny fraction of parameters trains, and adapters can be merged or swapped at serving time.
- **QLoRA:** load the frozen base model in 4-bit (NF4) and train LoRA adapters on top, making 7B-class fine-tuning possible on a single consumer or Colab GPU.

- Others: adapters, prefix and prompt tuning, DoRA.

Data matters most. A few hundred to a few thousand high-quality, diverse examples often beat tens of thousands of noisy ones. Format examples in the model's chat template, deduplicate, hold out a test set, check for leakage, and verify any synthetic data.

```
import torch
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig

bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type='nf4',
                         bnb_4bit_compute_dtype=torch.bfloat16)
model = AutoModelForCausalLM.from_pretrained(MODEL_ID,
quantization_config=bnb)
lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                  target_modules='all-linear', task_type='CAUSAL_LM')
trainer = SFTTrainer(
    model=model, train_dataset=train_ds, eval_dataset=eval_ds,
peft_config=lora,
    args=SFTConfig(output_dir='out', per_device_train_batch_size=4,
                   gradient_accumulation_steps=4, learning_rate=2e-4,
                   num_train_epochs=2, bf16=True, logging_steps=10))
trainer.train()
```

Practical guidance: LoRA learning rates around 1e-4 to 2e-4, one to three epochs, and watch validation loss and task metrics for overfitting. Beware catastrophic forgetting of general ability. Follow SFT with DPO when you have preference data. For new languages or specialized domains, continued pretraining on raw domain text (and sometimes vocabulary extension) comes before SFT. Hosted fine-tuning APIs exist when you do not want to manage GPUs.

Lab 11. QLoRA fine-tune a 1B-8B open model on a structured-extraction task. Compare it against your best prompt-only baseline on a held-out set, with accuracy, cost and latency.

Self-check: Why does LoRA work with so few trainable parameters? When would you choose RAG over fine-tuning?

Module 12: Evaluation

Goals: measure quality rigorously. This is the skill that separates demos from products.

Kinds of evaluation:

- **Public benchmarks** (MMLU, GSM8K, HumanEval, SWE-bench and many others) compare models in general, but they saturate and can be contaminated by training data. Never choose a model on leaderboards alone.
- **Task-specific eval sets:** 50-500 real, representative examples with expected answers or grading rubrics. This is your most valuable asset.
- **Programmatic checks:** exact match, regex, schema validation, unit tests for generated code.
- **LLM-as-judge:** a strong model grades outputs against a rubric or compares two answers. Known biases include preferring longer answers, preferring the first position, and favoring its own style. Calibrate the judge against human labels before trusting it.
- **Human evaluation** for subjective quality and for calibrating automated metrics.
- **Classic metrics** (accuracy, F1, BLEU, ROUGE, BERTScore) are useful only where their assumptions fit; overlap metrics correlate poorly with quality for open-ended text.

Eval-driven development. Build the eval first. Change one thing at a time (prompt, model, retriever, temperature). Run the suite automatically in CI so regressions are caught before release, and track cost and latency next to quality.

Statistical care. Outputs vary between runs, so use enough examples, report confidence intervals, and do not celebrate a 1-point gain on 30 examples.

In production: sample real traces for review, collect explicit feedback (thumbs up or down) and implicit signals (edits, retries, abandonment), and run A/B tests for significant changes.

Red teaming. Systematically attack your own system with adversarial prompts, jailbreaks, injection attempts and sensitive-topic probes, and add the failures to your regression suite.

Tooling examples: lm-evaluation-harness, Inspect, promptfoo, RAGAS, and observability platforms such as LangSmith, Braintrust and Langfuse.

Lab 12. Build a 100-example evaluation set for your Module 8 or Module 9 project, write an LLM judge with a rubric, measure its agreement with your own labels, and run the suite in GitHub Actions on every pull request.

Self-check: Why can a model with a higher benchmark score be worse for your task? What biases must you correct for in an LLM judge?

Part 3: Production and Frontier

Module 13: Inference Optimization and Serving

Goals: serve models fast and cheaply, and size hardware correctly.

Two phases of generation. *Prefill* processes the whole prompt in parallel and is compute-bound. *Decode* generates one token at a time and is limited by memory bandwidth, because every step reads all the weights and the KV cache. Most optimizations target decode.

Metrics to track: time to first token (TTFT), time per output token, tokens per second per user and per GPU, tail latency (p95, p99), and cost per million tokens.

Memory math.

- Weights: parameters x bytes per parameter. A 7B model in 16-bit needs about 14 GB; in 4-bit about 4 GB.
- KV cache per sequence: `2 x layers x kv_heads x head_dim x tokens x bytes_per_value`. It grows with context length and batch size, and often limits how many users one GPU can serve.

Quantization. Store weights (and sometimes activations or the KV cache) in fewer bits: BF16 to FP8, INT8 or INT4. Post-training methods such as GPTQ and AWQ, the GGUF format used by llama.cpp, and bitsandbytes give large memory savings with small quality loss when done well. Always re-run your own evaluation after quantizing.

Serving techniques:

- **Continuous batching:** add and remove requests from the running batch at every step instead of waiting for a full batch to finish.
- **PagedAttention** (vLLM): manage the KV cache in small blocks like virtual memory, cutting waste.
- **Prefix caching:** reuse the KV cache for shared system prompts and documents.
- **Speculative decoding:** a small draft model proposes several tokens and the large model verifies them in one pass, giving speedups with identical output quality.
- **Chunked prefill, optimized kernels (FlashAttention), tensor-parallel serving** for models too large for one GPU.

Engines: vLLM, SGLang, TensorRT-LLM, Hugging Face TGI for servers; llama.cpp and Ollama for local and edge use; managed endpoints from cloud providers.

Build or buy. Use a hosted API when volume is low to moderate and you want the strongest models and no operations burden. Self-host when you have high steady volume, strict privacy or data-residency needs, need custom fine-tuned models, or need predictable unit costs. Do the arithmetic with real traffic numbers before deciding.

Capacity planning. Estimate requests per second, average input and output lengths, and your latency target, then load test with realistic traffic rather than a single request in a loop.

Lab 13. Serve a quantized 7B-8B model with vLLM or llama.cpp. Load test at several concurrency levels and plot TTFT and throughput. Compare 16-bit against 4-bit on both speed and your task accuracy.

Self-check: Why is decode memory-bound? Why does continuous batching raise throughput? How big is the KV cache for 32 concurrent 8k-token requests on your model?

Module 14: LLMOps and Production Architecture

Goals: turn a prototype into a reliable, observable, cost-controlled service.

Typical architecture, in request order:

1. Client and API gateway (authentication, rate limiting).
2. Application or orchestration layer (prompt assembly, retrieval, tool calls, guardrails).
3. Model gateway (model routing, fallbacks, caching, retries).
4. Models (hosted APIs and/or self-hosted).
5. Supporting services: vector database, cache, queue for long jobs, secrets manager.
6. Observability, evaluation pipeline and a feedback store alongside everything.

Observability. Trace every request end to end: the final prompt, retrieved chunks, tool calls, outputs, token counts, latency and cost, tied to a user or session ID. Redact PII in logs. Build dashboards and alerts for error rate, latency, cost per request and quality signals. OpenTelemetry-based tracing and platforms such as Langfuse or LangSmith help.

Cost control:

- Route easy requests to small models and hard ones to large models (cascades).
- Cache: exact-match, semantic caching where safe, and provider prompt caching for long shared prefixes.
- Trim prompts, limit output length, and use batch endpoints for offline work.
- Set per-user and per-tenant budgets to avoid runaway bills.

Reliability: timeouts, retries with backoff, provider fallbacks, circuit breakers, graceful degradation (for example, return retrieved sources if generation fails), and queues for long-running agents. Pin exact model versions and run your evaluation suite before upgrading, because model behavior changes between versions and old versions get retired.

Guardrails. Validate inputs (length, type, injection heuristics), validate outputs (schema, banned content, PII, groundedness checks), and apply moderation where the product requires it.

Delivery process. Version prompts, configs and model choices together. Gate releases on evaluation results in CI. Use shadow traffic, canary releases and fast rollback. Keep a record of what changed when quality moves.

The data flywheel. Production traces lead to reviewed failures, which lead to new evaluation cases and training data, which lead to improved prompts, retrieval or fine-tuned models.

UX for latency. Stream tokens, show progress for agents, and run slow work in the background with notifications.

Lab 14. Wrap your Module 9 RAG app in a FastAPI service with a streaming endpoint, structured logging and tracing, caching, rate limiting and a budget guard. Containerize it with Docker, add a CI pipeline with an evaluation gate, and deploy it to a cloud platform.

Self-check: What would you alert on for an LLM service that you would not alert on for a normal API? Why pin model versions?

Module 15: Safety, Security, Ethics and Governance

Goals: build systems that are safe to put in front of real users.

Hallucination. Models produce plausible text, not guaranteed truth. Mitigate with retrieval and citations, allowing the model to abstain, verification steps (a second pass or tool check), constrained tasks, and human review for high-stakes use. Measure it; do not assume it is solved.

Prompt injection. Attackers hide instructions in user input (direct) or in content the model reads such as web pages, emails and documents (indirect). There is currently no complete model-level fix, so use defense in depth:

- Least privilege: give the model and its tools only the access the task needs.
- Keep secrets out of prompts; never let the model hold credentials.
- Treat all retrieved and tool-returned text as untrusted data.
- Validate and sanitize outputs before using them (to prevent injection into HTML, SQL or shell).
- Require human confirmation for sensitive actions; use allow-lists and sandboxes.
- Monitor and log for abuse.

Other risks (see the OWASP Top 10 for LLM Applications): jailbreaks, sensitive-data leakage, excessive agency, insecure output handling, training-data or retrieval poisoning, supply-chain risk (prefer the safetensors format over pickle files and verify model sources), and denial-of-wallet attacks that burn your budget.

Privacy and compliance. Know what data you send to a provider, its retention terms, and whether you need self-hosting. Minimize and redact personal data. Understand the regulations that apply to you, such as GDPR, India's DPDP Act and sector rules, and the EU AI Act's risk-based approach if you serve those markets. Get legal advice for real deployments.

Bias and fairness. Evaluate performance across demographic groups, dialects and languages. Low-resource languages, including many Indic languages and code-mixed text such as Hinglish, often perform worse, so test them explicitly if your users speak them.

Licensing and copyright. 'Open weights' is not the same as open source. Read the model license (commercial use, size or user thresholds, acceptable-use rules) and dataset licenses before shipping.

Transparency and oversight. Tell users when they are talking to AI, document your system (a model or system card: purpose, data, limits, evaluation results), and keep humans in the loop where errors are costly.

Lab 15. Red-team your own RAG or agent app with at least 20 attacks (direct and indirect injection, data-leak attempts, harmful requests). Implement three defenses, re-run the attacks, and write a one-page system card with before-and-after results.

Self-check: Why is indirect prompt injection harder to stop than direct injection? What does least privilege mean for an email-reading agent?

Module 16: Advanced and Frontier Topics

Goals: be able to read current research and extend what you have built.

Multimodal models. Vision-language models connect an image encoder (such as a Vision Transformer) to an LLM through a projection layer, so images become tokens the LLM can attend to. Speech uses ASR and TTS pipelines or natively audio-capable models. Applications include document and chart understanding, OCR-free parsing, and multimodal RAG. Diffusion models are the other major family, dominant for image and video generation.

Long context and context engineering. Long windows are powerful but costly, and models can miss information buried in the middle. Decide between stuffing context, RAG and summarization based on cost, latency and accuracy. Context engineering means deliberately choosing what goes into the window at each step of an agent: instructions, relevant memory, retrieved facts, and tool results.

Reasoning and test-time compute. Instead of only making models bigger, spend more computation at inference: longer reasoning traces, many samples with majority vote or best-of-N selection using a verifier, and search over reasoning steps. RL with verifiable

rewards trains models to use that budget well. Practical questions are when reasoning is worth its latency and cost, and how to cap it.

Efficient and alternative architectures. MoE for capacity at low active compute, state-space models such as Mamba and hybrid attention-SSM designs for linear-time sequence processing, linear attention variants, diffusion-style language models (still emerging), small language models for on-device use, distillation, and model merging.

Interpretability. Probing, attention and activation analysis, sparse autoencoders for discovering features, and circuit analysis. This field aims to explain why models behave as they do and to support safety.

Frontier training and agents. Synthetic data and self-improvement loops, RL scaling, continual learning, computer-use and coding agents, long-horizon tasks, multi-agent systems, and benchmarks that measure real work such as software engineering tasks.

Research skills. Read a paper in three passes (abstract and figures; method; details and appendix), reproduce one result, and keep a notes file. See the reading list at the end.

Lab 16 (choose one): (a) add RoPE and grouped-query attention to your Module 4 model and measure the effect; (b) build a tiny vision-language model by connecting a frozen vision encoder to a small LLM; (c) implement best-of-N sampling with a verifier on a math dataset and plot accuracy against N.

Self-check: What does an image encoder plus projector do? When is best-of-N worth its cost?

Part 4: Capstone

Choose one project and deliver it to production standard.

Project options:

1. **Domain RAG assistant:** hybrid retrieval, reranking, citations, access control, and an evaluation harness, built over a real document set (college handbook, legal or policy texts, technical documentation).
2. **Fine-tuned small model:** QLoRA fine-tune an open model for a domain task (for example Hindi-English code-mixed extraction or summarization), quantize it, serve it with vLLM, and compare it with an API baseline on quality, cost and latency.
3. **Tool-using data analyst agent:** natural-language questions over CSV or SQL data, sandboxed code execution, chart generation, step limits, and a 50-task evaluation suite.
4. **Build an LLM from scratch:** pretrain a small model, then SFT and DPO it, evaluate it, and write a technical report on every stage.

5. **Support copilot:** guardrails, observability, cost controls and an offline-plus-online evaluation plan with a simulated A/B test.

Required deliverables: GitHub repository with a clear README; architecture diagram; evaluation report with a test set, metrics and error analysis; deployed API or demo; cost and latency analysis; safety review with red-team results; a 5-minute demo video or live presentation.

Grading rubric:

- Design and correctness: 25%
- Evaluation rigor: 25%
- Engineering quality (tests, CI, documentation): 20%
- Safety, security and cost awareness: 15%
- Communication and demo: 15%

14-Week Schedule

Plan about 4 hours of reading and lectures plus 6 hours of lab work per week.

- Week 1: Modules 1-2 (LLM overview, math prerequisites)
- Week 2: Module 3 (tokenization, embeddings, positions)
- Week 3: Module 4 (Transformer architecture)
- Week 4: Module 5 (pretraining and distributed training)
- Week 5: Module 6 (post-training and alignment)
- Week 6: Modules 7-8 (APIs and prompt engineering)
- Week 7: Module 9 (embeddings and RAG)
- Week 8: Module 10 (tool use and agents)
- Week 9: Module 11 (fine-tuning)
- Week 10: Module 12 (evaluation); choose capstone and write its proposal
- Week 11: Module 13 (inference and serving)
- Week 12: Module 14 (LLMOps); capstone build begins in earnest
- Week 13: Modules 15-16 (safety, frontier); capstone red-teaming and evaluation
- Week 14: capstone polish, demos, peer review

Industry-Readiness Checklist

You are ready for an LLM engineering role when you can honestly say yes to each of these:

- I can implement attention and a small Transformer from memory and explain every tensor shape.
- I can explain tokenization, positional encoding, KV caching and why decoding is memory-bound.
- I can describe the pretraining, SFT, preference-optimization and RL stages and what each fixes.
- I can build a RAG system with hybrid retrieval and measure retrieval and answer quality separately.
- I can build a tool-using agent with step limits, validation and logging.
- I can fine-tune with LoRA or QLoRA, and I know when not to fine-tune.
- I always build an evaluation set before optimizing, and I can calibrate an LLM judge.
- I can quantize and serve a model, and estimate memory, latency and cost.
- I can deploy a monitored, cost-controlled LLM service with CI evaluation gates.
- I can threat-model an LLM application and apply layered defenses.

Interview Preparation

Practice answering these aloud, with a whiteboard sketch where useful:

1. Walk through what happens from a prompt to the first generated token.
2. Why is attention quadratic, and how do FlashAttention, GQA and sliding windows help?
3. Explain LoRA and why it works. When would you use QLoRA?
4. RAG versus fine-tuning versus long context: how do you choose?
5. Your RAG system gives wrong answers. How do you debug it step by step?
6. How would you evaluate a customer-support chatbot before and after launch?
7. Estimate the GPU memory to serve a 70B model to 50 concurrent users.
8. How do you defend an email-summarizing agent against prompt injection?
9. Explain RLHF and DPO, and the trade-offs between them.
10. Design a cost-reduction plan for an LLM feature whose bill tripled.

Reference Reading and Resources

Core papers: Attention Is All You Need; BERT; Language Models are Few-Shot Learners (GPT-3); Scaling Laws for Neural Language Models; Training Compute-Optimal Large Language Models (Chinchilla); Training Language Models to Follow Instructions with Human Feedback (InstructGPT); Direct Preference Optimization; Constitutional AI;

RoFormer (RoPE); FlashAttention; Efficient Memory Management for LLM Serving with PagedAttention (vLLM); LoRA; QLoRA; Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks; Chain-of-Thought Prompting; ReAct; Mamba.

Courses and books: Stanford CS224N (NLP with deep learning) and CS336 (Language Modeling from Scratch); Andrej Karpathy's Neural Networks: Zero to Hero series and nanoGPT; the Hugging Face LLM and NLP courses; *Build a Large Language Model (From Scratch)* by Sebastian Raschka; *AI Engineering* and *Designing Machine Learning Systems* by Chip Huyen; Anthropic's 'Building effective agents' guide and the prompt-engineering documentation of your chosen provider.

Habits that build careers: keep a public GitHub portfolio with documented projects and evaluation results, write short technical posts about what you learned, reproduce one paper per month, and read provider and open-source release notes regularly because this field changes quickly.